

SE3082

AWS EC2 Lab - Screenshots and Calculations

Lab 06

Exercise 7 : Parallelizing the mandabolt set code

IT24610328

On instance with 2 vCPUs (t3.micro)

Nano - creating the parallelized mandabolt set code

```
GNU nano 6.2                                ex7_mandelbrot.c

# define NPOINTS 2000
# define MAXITER 2000

struct complex{
    double real;
    double imag;
};

int main(){
    int i, j, iter, numoutside = 0;
    double area, error, ztemp;
    struct complex z, c;
    double tstart, tstop, tcalc;

    tstart = omp_get_wtime();

    #pragma omp parallel for private(j, iter, c, z, ztemp) reduction(+:numoutside)
    for (i=0; i<NPOINTS; i++) {
        for (j=0; j<NPOINTS; j++) {
            c.real = -2.0+2.5*(double)(i)/(double)(NPOINTS)+1.0e-7;
            c.imag = 1.125*(double)(j)/(double)(NPOINTS)+1.0e-7;
            z=c;
            for (iter=0; iter<MAXITER; iter++){
                ztemp=(z.real*z.real)-(z.imag*z.imag)+c.real;
                z.imag=z.real*z.imag*2+c.imag;
                z.real=ztemp;
                if ((z.real*z.real+z.imag*z.imag)>4.0e0) {
                    numoutside++;
                    break;
                }
            }
        }
    }

    area=2.0*2.5*1.125*(double)(NPOINTS*NPOINTS-numoutside)/(double)(NPOINTS*NPOINTS);
    error=area/(double)NPOINTS;

    tstop = omp_get_wtime();
    tcalc = tstop - tstart;

    printf("Threads used: %d\n", omp_get_max_threads());
    printf("Area of Mandelbrot set = %12.8f +/- %12.8f\n",area,error);
    printf("Time = %f seconds\n", tcalc);

    return 0;
}
```

Outputs of running the parallized mandabolt set code

Running with one thread

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex7_mandelbrot.o  
Threads used: 1  
Area of Mandlebrot set = 1.50928453 +/- 0.00075464  
Time = 16.337244 seconds
```

Running with two threads

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex7_mandelbrot.o  
Threads used: 2  
Area of Mandlebrot set = 1.50928453 +/- 0.00075464  
Time = 15.530174 seconds
```

Running with three threads

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex7_mandelbrot.o  
Threads used: 3  
Area of Mandlebrot set = 1.50928453 +/- 0.00075464  
Time = 13.858289 seconds
```

Running with four threads

```
[ec2-user@ip-172-31-17-201 ~]$ ./ex7_mandelbrot.o  
Threads used: 4  
Area of Mandlebrot set = 1.50928453 +/- 0.00075464  
Time = 11.896613 seconds
```

Correctness

All 4 runs give the exact identical area (1.50928453), confirming the reduction is working correctly regardless of thread count.

Timing differences

Although this instance only has 2 physical vCPUs, requesting 3 or 4 threads still improved performance (16.3s to 15.5s to 13.9s to 11.9s). This is because the Mandelbrot calculation has highly uneven work per grid point (points near the set boundary take far more iterations to resolve than points that diverge quickly). With only 2 threads, OpenMP's static scheduling divides the 2000 outer iterations into 2 large fixed chunks, so if one chunk happens to contain more computationally expensive points, that thread becomes a bottleneck while the other sits idle. Using more threads than physical cores shrinks the chunk size assigned to each thread, better distributing the uneven workload even though the OS still has to time-slice execution across only 2 real cores, resulting in improved load balancing rather than genuine additional parallel throughput.